

Lab 05: GPG Man in the Middle (MITM) Attack Exercise Report

Exercise Goal

Understand that a public key by itself does not guarantee trust. MITM attacks can happen during key exchange, so fingertip (fingerprint) verification is essential. The Web of Trust helps detect and limit such attacks.

1. Key Generation

```
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: directory '/root/.gnupg' created
gpg: keybox '/root/.gnupg/pubring.kbx' created
Please select what kind of key you want:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysizes do you want? (3072) 4096
Requested keysizes is 4096 bits
Please specify how long the key should be valid.
  0 = key does not expire
  <n> = key expires in n days
  <n>w = key expires in n weeks
  <n>m = key expires in n months
  <n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Friday 28 May 2027 12:01:39 PM IST
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Alice
Email address: alice@example.com
Comment:
You selected this USER-ID:
  "Alice <alice@example.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
```

```
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /root/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/root/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/root/.gnupg/openpgp-revocs.d/ACE45BA5C00C420483142064C59146A6D15B6A43.rev'
public and secret key created and signed.

pub  rsa4096 2026-05-28 [SC] [expires: 2027-05-28]
    ACE45BA5C00C420483142064C59146A6D15B6A43
uid                Alice <alice@example.com>
sub  rsa4096 2026-05-28 [E] [expires: 2027-05-28]
```

```

L-$ sudo gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) +default+
  (10) ECC (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
    0 = key does not expire
    <n> = key expires in n days
    <n>w = key expires in n weeks
    <n>m = key expires in n months
    <n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Friday 28 May 2027 12:04:22 PM IST
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Bob
Email address: bob@example.com
Comment:
You selected this USER-ID:
    "Bob <bob@example.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/root/.gnupg/openpgp-revocs.d/2EF624FF3F12A764BF9B05B184ADE366B10BC1AC.rev'
public and secret key created and signed.

```

```

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/root/.gnupg/openpgp-revocs.d/2EF624FF3F12A764BF9B05B184ADE366B10BC1AC.rev'
public and secret key created and signed.

pub  rsa4096 2026-05-28 [SC] [expires: 2027-05-28]
    2EF624FF3F12A764BF9B05B184ADE366B10BC1AC
uid                               Bob <bob@example.com>
sub  rsa4096 2026-05-28 [E] [expires: 2027-05-28]

```

```

└─$ sudo gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
 (10) ECC (sign only)
 (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
  0 = key does not expire
<n> = key expires in n days
<n>w = key expires in n weeks
<n>m = key expires in n months
<n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Friday 28 May 2027 12:06:17 PM IST
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Mallory
Email address: mallory@example.com
Comment:
You selected this USER-ID:
  "Mallory <mallory@example.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/root/.gnupg/openpgp-revocs.d/E3AFF5CE76CDB2F351FB2299288CA9C073CFB5C9.rev'
public and secret key created and signed.

```

```

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/root/.gnupg/openpgp-revocs.d/E3AFF5CE76CDB2F351FB2299288CA9C073CFB5C9.rev'
public and secret key created and signed.

pub   rsa4096 2026-05-28 [SC] [expires: 2027-05-28]
       E3AFF5CE76CDB2F351FB2299288CA9C073CFB5C9
uid     [ultimate] Mallory <mallory@example.com>
sub     rsa4096 2026-05-28 [E] [expires: 2027-05-28]

```

2. Verification

```

└─$ sudo gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   3  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 3u
gpg: next trustdb check due at 2027-05-28
/root/.gnupg/pubring.kbx

pub   rsa4096 2026-05-28 [SC] [expires: 2027-05-28]
       ACE45BA5C00C420483142064C59146A6D15B6A43
uid     [ultimate] Alice <alice@example.com>
sub     rsa4096 2026-05-28 [E] [expires: 2027-05-28]

pub   rsa4096 2026-05-28 [SC] [expires: 2027-05-28]
       2EF624FF3F12A764BF9B058184ADE366810BC1AC
uid     [ultimate] Bob <bob@example.com>
sub     rsa4096 2026-05-28 [E] [expires: 2027-05-28]

pub   rsa4096 2026-05-28 [SC] [expires: 2027-05-28]
       E3AFF5CE76CDB2F351FB2299288CA9C073CFB5C9
uid     [ultimate] Mallory <mallory@example.com>
sub     rsa4096 2026-05-28 [E] [expires: 2027-05-28]

```

3. Fingerprint Printing

```
└─$ sudo gpg --fingerprint alice@example.com
pub  rsa4096 2026-05-28 [SC] [expires: 2027-05-28]
    ACE4 5BA5 C00C 4204 8314  2064 C591 46A6 D15B 6A43
uid          [ultimate] Alice <alice@example.com>
sub  rsa4096 2026-05-28 [E] [expires: 2027-05-28]
```

```
└─$ sudo gpg --fingerprint bob@example.com
pub  rsa4096 2026-05-28 [SC] [expires: 2027-05-28]
    2EF6 24FF 3F12 A764 BF9B  05B1 84AD E366 B10B C1AC
uid          [ultimate] Bob <bob@example.com>
sub  rsa4096 2026-05-28 [E] [expires: 2027-05-28]
```

```
└─$ sudo gpg --fingerprint mallory@example.com
pub  rsa4096 2026-05-28 [SC] [expires: 2027-05-28]
    E3AF F5CE 76CD B2F3 51FB  2299 288C A9C0 73CF B5C9
uid          [ultimate] Mallory <mallory@example.com>
sub  rsa4096 2026-05-28 [E] [expires: 2027-05-28]
```

4. MITM Attack And Mallory Decrypts Message

```
└─$ sudo gpg --armor --export mallory@example.com > bob_pubkey.asc
```

```
(kali㉿kali)-[~]
└─$ gpg --armor --export mallory@example.com > bob_pubkey.asc

(kali㉿kali)-[~]
└─$ gpg --import bob_pubkey.asc
gpg: key 6899992862EC8516: "Mallory <mallory@example.com>" not changed
gpg: Total number processed: 1
gpg:      unchanged: 1

(kali㉿kali)-[~]
└─$

(kali㉿kali)-[~]
└─$ echo "Confidential message for Bob" > secret.txt

(kali㉿kali)-[~]
└─$ gpg --encrypt --recipient bob@example.com secret.txt

(kali㉿kali)-[~]
└─$ gpg --decrypt secret.txt.gpg
gpg: encrypted with rsa4096 key, ID 75A658D4F852930F, created 2026-05-28
    "Bob <bob@example.com>"
Confidential message for Bob
```

5. Web of Trust

```

(kali㉿kali)-[~]
└─$ gpg --edit-key bob@example.com
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Secret key is available.

sec  rsa4096/26A26D39087E3884
     created: 2026-05-28  expires: 2027-05-28  usage: SC
     trust: ultimate      validity: ultimate
ssb  rsa4096/75A658D4F852930F
     created: 2026-05-28  expires: 2027-05-28  usage: E
[ultimate] (1). Bob <bob@example.com>

```

```

gpg> trust
sec  rsa4096/26A26D39087E3884
     created: 2026-05-28  expires: 2027-05-28  usage: SC
     trust: ultimate      validity: ultimate
ssb  rsa4096/75A658D4F852930F
     created: 2026-05-28  expires: 2027-05-28  usage: E
[ultimate] (1). Bob <bob@example.com>

Please decide how far you trust this user to correctly verify other users' keys
(by looking at passports, checking fingerprints from different sources, etc.)

  1 = I don't know or won't say
  2 = I do NOT trust
  3 = I trust marginally
  4 = I trust fully
  5 = I trust ultimately
  m = back to the main menu

Your decision? 5
Do you really want to set this key to ultimate trust? (y/N) y

sec  rsa4096/26A26D39087E3884
     created: 2026-05-28  expires: 2027-05-28  usage: SC
     trust: ultimate      validity: ultimate
ssb  rsa4096/75A658D4F852930F
     created: 2026-05-28  expires: 2027-05-28  usage: E
[ultimate] (1). Bob <bob@example.com>

gpg> quit

```

5. Additional Challenge: signing Bob's Key

```
(kali㉿kali)-[~]
$ gpg --sign-key bob@example.com

sec  rsa4096/26A26D39087E3884
     created: 2026-05-28  expires: 2027-05-28  usage: SC
     trust: ultimate    validity: ultimate
ssb  rsa4096/75A658D4F852930F
     created: 2026-05-28  expires: 2027-05-28  usage: E
[ultimate] (1). Bob <bob@example.com>

sec  rsa4096/26A26D39087E3884
     created: 2026-05-28  expires: 2027-05-28  usage: SC
     trust: ultimate    validity: ultimate
Primary key fingerprint: E64B 4080 746D B0FE 8298  AA0E 26A2 6D39 087E 3884

    Bob <bob@example.com>

This key is due to expire on 2027-05-28.
Are you sure that you want to sign this key with your
key "Alice <alice@example.com>" (78937C3F3A2A48E8)

Really sign? (y/N) y
```

Questions

1. Why doesn't GPG detect MITM attacks automatically?

GPG can encrypt messages, but it cannot automatically know if a public key really belongs to the correct person. An attacker can replace the real public key with a fake one, and GPG may not know the difference. GPG cannot check a person's real identity by itself. Because of this, we need to verify the key separately, for example by checking its fingerprint or using the Web of Trust. Without verification, we might encrypt a message using the attacker's key instead of the real person's key.

2. What is a fingerprint and why is it important?

A fingerprint is a unique code that represents a GPG public key. It is used to check whether the key is the correct one. It is important because we can compare the fingerprint with the key owner to make sure the key has not been changed or replaced. For example, we can check the fingerprint in person, over a phone call, or through another trusted method. This helps prevent MITM attacks.

3. Why is email not a secure channel for exchanging keys?

Email is not fully secure for exchanging keys because messages can be intercepted, changed, or fake. An attacker could replace a real public key with their own key. The receiver might then encrypt the message using the attacker's key without realizing it. Because of this, it is better to verify the key fingerprint separately instead of trusting a key just because it was received by email.

4. How does the Web of Trust reduce MITM risk?

The Web of Trust is a system where people verify and sign each other's public keys.

For example, if Alice trusts Carol and Carol has verified Bob's key, Alice can have more confidence that Bob's key is genuine.

When several trusted people sign a key, it becomes harder for an attacker to use a fake key without being noticed.

Still, the Web of Trust needs at least one direct verification, such as checking the fingerprint with the person, to create a strong starting point of trust.